JavaScript Event Loop Notes

JavaScript is single-threaded and synchronous by default. To handle asynchronous tasks like timers, API calls, and promises, JavaScript uses the Event Loop.

Main Components of Event Loop

Component	Purpose
Call Stack	Executes functions one by one
Web APIs	Handles async operations like setTimeout
Callback Queue	Stores completed callback functions
Microtask Queue	Stores promises and microtasks
Event Loop	Moves tasks to Call Stack when empty

Basic Event Loop Example

```
console.log("Start");

setTimeout(() => {
    console.log("Inside Timeout");
}, 2000);

console.log("End");
```

Output:
Start
End
Inside Timeout

Explanation: 1. "Start" executes first. 2. setTimeout goes to Web APIs. 3. "End" executes immediately. 4. After 2 seconds, callback moves to Callback Queue. 5. Event Loop checks Call Stack. 6. If stack is empty, callback executes.

Promises and Microtask Queue

```
console.log("Start");

setTimeout(() => {
    console.log("Timeout");
}, 0);

Promise.resolve().then(() => {
    console.log("Promise");
});

console.log("End");
```

Output:
Start
End
Promise

Timeout

Promises are stored in the Microtask Queue. Microtasks have higher priority than the Callback Queue. Therefore, Promise executes before setTimeout.

Important Concepts

• JavaScript executes one task at a time.
• Event Loop continuously checks the Call Stack.
• Microtasks execute before normal callbacks.
• setTimeout does not guarantee exact timing.
• async/await internally uses Promises.

Useful Visualizers

1. Loupe Event Loop Visualizer
https://latentflip.com/loupe/

2. JS Visualizer 9000
https://www.jsv9000.app/

3. JavaScript.info Event Loop Guide
https://javascript.info/event-loop